

From a fly wiring map to an experiment you can understand

Neuroscience from first principles. Real dataset examples. Every dashboard control. CPU and GPU setup. Evidence, limitations and a staged research plan.

UNDERSTAND THE BIOLOGY

TRACE THE DATA

CONTROL THE SIMULATION

Edition 1.0 | 19 September 2026 | 64 pages

An exploratory software model. No demonstrated learning or validated fly behavior.

A guide to the book

Click a topic or use your PDF reader's bookmarks.

Start here / What you have built	5
Start here / How to read the evidence	6
01 Biology / A neuron in plain language	7
01 Biology / Synapses, excitation and inhibition	8
01 Biology / The parts of the fly nervous system	9
01 Biology / L1, T4 and T5: names you will see	10
01 Biology / Eyes, columns and screens	11
01 Biology / How a brain becomes a wiring map	12
01 Biology / What is open and what is missing	13
02 Dataset / The files we actually download	14
02 Dataset / Real neuron rows, explained	15
02 Dataset / A row as code, and missing values	16
02 Dataset / Neurotransmitters are annotations	17
02 Dataset / Real connections become modeled weights	18
02 Dataset / From raw segments to retained neurons	19
02 Dataset / Arrays, sparse graphs and integrity	20
03 Simulation / What changes on every tick	21
03 Simulation / The equations we actually execute	22
03 Simulation / Parameters, rates and time units	23
03 Simulation / Why model stability is a scientific gate	24

Contents continued

Click a topic or use your PDF reader's bookmarks.

04 Dashboard / A screen you can read left to right	25
04 Dashboard / The three-panel workspace	26
04 Dashboard / Run, Pause, Step, Reset and Save	27
04 Dashboard / Pattern choices and the input image	28
04 Dashboard / Every input slider and selector	29
04 Dashboard / Drawing and placing a spot	30
04 Dashboard / Manual neural pulses	31
04 Dashboard / Reading the circuit view	32
04 Dashboard / Response bars, history and spike events	33
04 Dashboard / Neuron inspection and saved sessions	34
05 Exercises / First exercise: establish a blank baseline	35
05 Exercises / Second exercise: a moving pattern	36
05 Exercises / Third exercise: paint, probe and pulse	37
06 Research / What scientific models have established	38
06 Research / Games and robots: read the whole system	39
06 Research / Language, pretrained AI and adapters	40
06 Research / Reward, memory and learning are different	41
06 Research / Mushroom bodies and a future experiment	42
06 Research / Experiment 1 and the staged roadmap	43
07 Installation / Choose a device and an execution mode	44

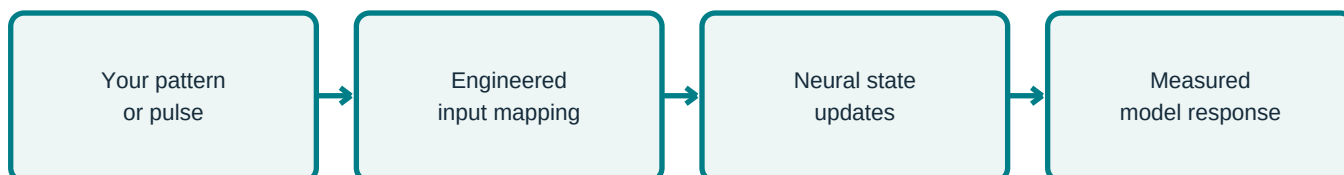
Contents continued

Click a topic or use your PDF reader's bookmarks.

07 Installation / A first installation that starts the UI	45
07 Installation / Run the real data or select CUDA	46
07 Installation / What setup.sh changes and preserves	47
07 Installation / Mac display, remote computation	48
07 Installation / Code map and reproducible records	49
07 Installation / What has actually been tested	50
08 Help / Installation and connection troubleshooting	51
08 Help / Input and result troubleshooting	52
08 Help / Frequently asked questions: biology	53
08 Help / Frequently asked questions: the software	54
08 Help / Frequently asked questions: claims and plans	55
09 Reference / Glossary: cells, circuits and data	56
09 Reference / Glossary: dynamics and experiments	57
09 Reference / Glossary: reproducibility and statistics	58
09 Reference / Additional annotation fields	59
10 Sources / References 1	60
10 Sources / References 2	61
10 Sources / References 3	62
10 Sources / References 4	63
10 Sources / Evidence, revisions and open questions	64

What you have built

FruitFlyBrain is a laboratory workbench for exploring a numerical model based on a fruit fly wiring map. You provide an input, advance a simulation and inspect its calculated response. The software is useful even when the response is zero, surprising or biologically implausible: those observations tell us which assumptions need investigation.



The implemented loop. There is no trained policy, reward signal or simulated fly body.

Two datasets serve different purposes. The **MaleCNS mode** uses the retained graph imported from a published male central nervous system reconstruction. The **portable demo** is an invented 256-node network for checking installation and learning the controls. Its positions and L1/T4/T5 labels are teaching placeholders. It must never be reported as a biological experiment.

What this book lets you do

You can read the interface without knowing neuroscience, inspect real source-table examples, understand the equations, install the project, run short exploratory tests and explain what the results establish. The book is a detailed reference to this version, not a claim to answer every open question in neuroscience.

Status on 19 September 2026: interactive exploration works; learning is off. Biological visual mapping, model stability, a connectivity control and a frozen evaluation protocol remain outstanding. Experiment 1 is a planned study, not a completed behavioral result.

How to read the evidence

Label	Meaning in this project
Anatomy	Published neuron objects, annotations and contact counts. Reconstruction and annotation can still contain uncertainty.
Prediction	A label inferred from data, such as neurotransmitter identity. Confidence is not a guarantee of physiological effect.
Model assumption	Chosen equations, time constants, signs, gain and reset rules. These are not all measured for each fly cell.
Engineering interface	Mapping a drawn pixel to a neuron, or a neuron to an action. Our current interface stops at neural measurements.
Observation	A number actually recorded from this implementation and specified input.
Hypothesis	An explanation or future prediction that needs a controlled test.

For a first reading, start with the biology and dashboard chapters. Then follow the worked exercises. The data and equations chapters explain how to audit what you see. The later research chapter shows how this work differs from an AI model trained to solve a game.

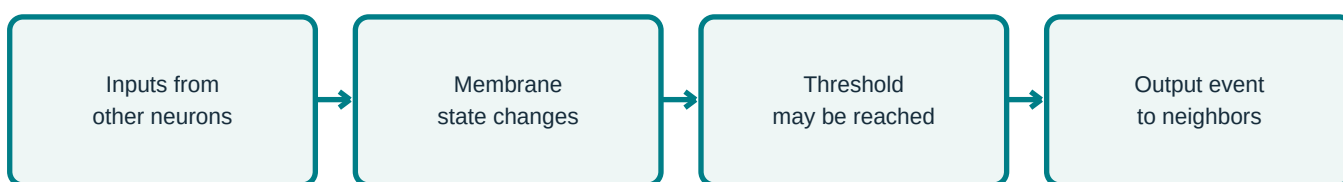
Use the clickable contents and PDF bookmarks to return to a term or control. Bracketed numbers point to the reference list. Exact example rows are also stored in docs/research/evidence/dataset_samples.json, including raw-file hashes and the extraction date.

The scientific plan and implementation can evolve. This handbook supersedes the initial overview; exp_1.pdf retains the detailed prospective protocol and its dated amendment. Future reward learning requires a separate exp_2.pdf.

A neuron in plain language

A neuron is a cell that receives signals, changes its internal electrical state and influences other cells. The **soma** is its cell body. Branches distribute inputs and outputs; simplified textbook drawings often call receiving branches dendrites and the long output branch an axon. In fly neurons, the precise arrangement is more varied than that cartoon. [13,14]

An electrical difference exists across the cell membrane. This is the **membrane potential**, measured in millivolts (mV). A negative value is a difference between inside and outside, not a negative amount of electricity. Real cells use ion channels, pumps and chemical processes. Our model represents a cell with a few numerical variables instead.



A teaching abstraction. The real cell has much richer physiology.

A **spike** or action potential is a brief electrical event. Some neurons also communicate through graded changes rather than behaving like identical on/off switches. Replacing every visual cell with a spiking proxy is a major approximation. The dashboard displays events produced by our equations, not recordings from living tissue.

Read the inspector

When the inspector shows -52 mV, that is the model state of the selected cell. If it shows 2 spikes/chunk, the cell crossed our model threshold twice during the most recent calculation window. Neither value tells you that the fly understood a shape or chose an action.

Synapses, excitation and inhibition

A **synapse** is a signaling connection between cells. At a chemical synapse, the presynaptic cell releases transmitter that acts on the postsynaptic cell. Its effect depends on receptors and the receiving cell, not only on the transmitter name. Synaptic signals can make a later spike more or less likely. [14]

Direction matters: **pre** means sender; **post** means receiver. A connection from A to B does not imply an equally strong connection from B to A. Two neurons may meet at many synaptic contacts. An aggregated connection row combines those contacts into one directed edge.

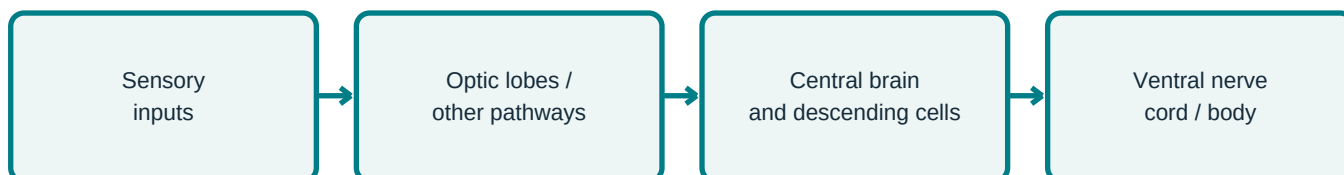
Term	Everyday reading	In our implementation
Excitation	Input tending to raise activity	Positive modeled fast weight
Inhibition	Input tending to suppress activity	Negative modeled fast weight
Contact count	How many reconstructed contacts connect a pair	Anatomical integer kept separately
Effective weight	How strongly an event changes the model	Contact count x 0.275 x assumed sign

For example, a published row with 32 contacts is not a measurement of 8.8 mV. Our software creates 8.8 by multiplying 32 by its chosen 0.275 scale. A different model can use the same anatomy with different dynamics and get different responses.

Zero effective weight does not mean the anatomical edge is absent. This version retains such edges but assigns no fast effect to unknown or modulatory transmitter categories. It does not implement their slower biology.

The parts of the fly nervous system

The **central nervous system (CNS)** includes the brain and the ventral nerve cord. The optic lobes are major visual-processing regions beside the central brain. The nerve cord contains circuits associated with the body; descending neurons convey signals toward it. An ascending pathway carries information in the opposite broad direction. These descriptions locate pathways; they do not assign a particular game button to a neuron. [1,2]



Conceptual route only. The actual graph is recurrent and has many parallel pathways.

The interface has six coarse population summaries: Visual input, Optic lobe, Visual projection, Central brain, Descending, and Ventral / other. These are software groupings derived from annotation superclass prefixes. They are not six anatomically complete compartments and are not six distinct thoughts.

The **Brain populations** view hides the broad ventral/other class. **Whole CNS sample** includes it. **Visual populations** shows visual input, optic-lobe and visual-projection groups. Changing this filter changes the drawing, not which neurons are simulated.

The MaleCNS run includes all 166,700 retained nodes even when the screen displays only a few thousand points. A filter, a hidden point or a missing soma location never removes that neuron from the dynamics.

L1, T4 and T5: names you will see

L1 is a named lamina neuron type in the fly visual system, not “layer one” of an artificial neural network. **R1-R6** refers to photoreceptor types. **T4** and **T5** are families of visual neurons. Biological studies distinguish T4 responses to moving brightness increases and T5 responses to moving brightness decreases, with directional subtypes. A dark object has both a leading and trailing edge, so “T5 means dark object” is too simple. [9,10]

Label	What it denotes	What the dashboard does
L1	Visual lamina cell type	Receives our directly injected image drive where coordinates exist
T4a-d	T4 directional subtypes	Groups all type labels beginning T4 by soma side
T5a-d	T5 directional subtypes	Groups all type labels beginning T5 by soma side
Descending	A broad anatomical class	Reports population activity; no steering action assigned

In the real animal, intermediate cell types and pathways connect these populations. The connectome contains those intermediate cells. We have not replaced the visual system with four boxes. Nevertheless, using one generic spiking equation and an uncalibrated input can fail to reproduce the known biological roles.

A type label is not a validated response in our simulator. T4a activity does not by itself prove a particular screen direction was detected. The a-d subtype-to-direction conventions and optical mapping must be independently checked before directional evaluation.

Eyes, columns and screens

The left/right choice on the screen selects which annotated L1 population receives external drive. “Both” sends the **same image** to both sets of normalized column coordinates. It does not divide the image into a left half and a right half, create a stereo camera or simulate the optics of two eyes.

In the real graph, 1,767 of 1,776 L1 cells have usable coordinates: 875 left and 892 right. The remaining nine L1 cells stay in the simulation but receive no direct patterned input from this encoder.

An optic-lobe **column** is a spatial organization of visual circuitry. The source fields assignedOIHex1 and assignedOIHex2 provide column coordinates for some cells. Our encoder independently rescales each eye’s coordinate range to 0-1 in each dimension and samples a rectangular 32 x 24 image. A hexagonal coordinate system is therefore being used as a convenient input address, not a calibrated visual angle.

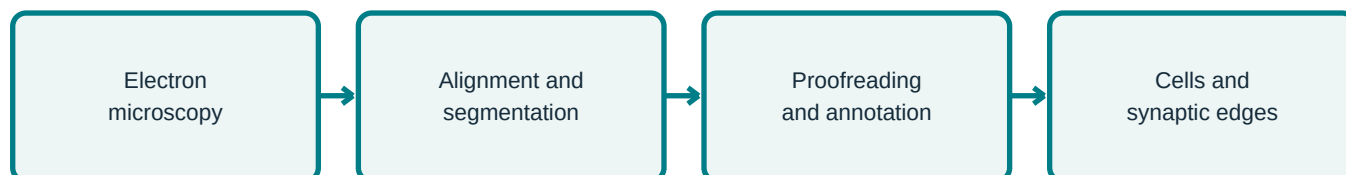
Space	What its coordinates mean
Screen image	Row and column of your drawn 32 x 24 pattern
L1 column map	Published column labels normalized separately for each side
Anatomical view	Soma/tosoma landmarks in reconstructed coordinates, rotated only for display
Physical fly vision	Would require optics, pose, angular mapping, receptor properties and calibration; not implemented

Left/right response groups use **somaSide**. That is the annotated side of a cell body, not necessarily the side of everything it senses or controls. Rotating the brain drawing does not rotate the input image. Changing the camera view is only a visualization action.

Moving a stripe rightward on this screen is a well-defined software input. Claiming that it corresponds to a particular natural fly-eye direction is a separate scientific step that remains unfinished.

How a brain becomes a wiring map

Connectomics begins with preserved tissue, electron microscopy and image reconstruction. Computers align imagery and segment structures; specialists correct reconstruction errors and annotate cells. Synapse detection links candidate sending and receiving structures. The resulting graph describes reconstructed connectivity. It is not a recording of the animal's voltage at the moment you run the software. [1,2]



A schematic reconstruction pipeline. Our project starts from its published data products.

Segmentation assigns image voxels to objects. A mistaken merge can combine branches from different cells; a split can separate parts of one cell. **Proofreading** corrects such problems. A named cell type and a transmitter prediction add useful information, but do not provide every receptor, channel, delay or plasticity rule needed for a physiological simulation.

Google's flood-filling-network repository provides a segmentation method, and Neuroglancer provides interactive visualization of large volumetric datasets. Those tools address reconstruction and inspection. They are not a downloaded pretrained fly agent. Our simulation code is an independent implementation built after the data release. [4,5]

"Complete" describes the scope of the reconstruction. It should not be read as error-free, complete molecular biology, complete lifetime experience or an exact behavioral emulator. For our experiments, the exact released files, inclusion policy and missing information matter more than a headline.

What is open and what is missing

Resource	Available use	Not supplied by that resource
MaleCNS data	Images, annotations, connectivity, synapse products and morphology through official services	A ready game-playing or language policy
Neuroglancer	Browse volumetric data and reconstructions	A neuronal dynamics solver
Flood-filling networks	Segmentation research code	A calibrated model of fly behavior
Supplemental analyses	Column assignments, cross-dataset mappings and other derived tables	Automatic correctness for our own encoder
FruitFlyBrain	Our importer, solvers, controls and run records	A trained mind or validated reward circuit

The dataset is a collaboration involving Janelia/FlyEM, Cambridge, the MRC Laboratory of Molecular Biology and Google Research. Credit the collaborators and original papers when using it. Dataset access and reuse terms are distinct from software-library licenses. [2,3,6]

Our implementation lacks a calibrated photoreceptor model, many cell-specific physiological properties, receptor-resolved synapses, a fly body, muscles, locomotor physics and validated feedback from action. No learned checkpoint arrives with the wiring graph. No language decoder or objective is implied by a neuron name.

You can run the equations without training. You cannot assume that untrained equations plus anatomy will solve an arbitrary task. Training an external readout, fitting physiological parameters and modifying synapses are three different operations.

The files we actually download

For this project the pinned MaleCNS v1.0 import uses three Feather files. Feather is a typed columnar binary format: it stores a table efficiently, with exact integer and missing-value information. It is not a CSV of floating-point AI weights. File URLs and SHA-256 digests are in `configs/datasets.json`; the download record stores the observed source metadata. [3]

File role	Observed rows	Contents
Body annotations	211,577	36 fields: identifiers, types, classes, sides, coordinates and curation labels
Body neurotransmitters	1,835,518	10 fields: segment IDs, predicted and consensus transmitter information
Segment connections	151,856,684	body_pre, body_post, weight; each a 64-bit integer

These are raw source-table sizes observed in the pinned files, not the retained neuron or edge counts. Raw tables include entries that do not all belong to our retained neuronal population. The importer joins records by exact identifiers and filters both endpoints of every connection.

The three downloads are approximately 1.1 GB in total; import uses additional RAM and disk. The prepared graph on the GX10 occupies about 406 MB, but that is not a memory budget for the whole preparation process. Sorting, joining, annotations and temporary arrays require more space.

The official release also exposes larger synapse-location tables, skeletons and microscopy volumes. Our default setup does not download them. You do not need the entire EM image volume to run this point-neuron model.

Real neuron rows, explained

The rows below were extracted from our checksum-verified source on 19 September 2026. They are real examples, not randomly invented IDs. “null” means unavailable in this field; it does not mean a coordinate of zero.

bodyId	type	side	hex1, hex2	status
10465	L1	L	28.0, 22.0	Traced
18251	L1	R	19.0, 32.0	Traced
13882	T4a	L	null, null	Traced
71174	T5a	R	null, null	Traced
10001	DNp01	R	null, null	Traced

Read the first row

10465 identifies a particular object in this release. Its cell-type label is L1; its soma side is L; its column labels are 28 and 22. Its somaLocation is null, even though it has usable column coordinates. It can receive patterned input without appearing at a recorded soma position in the anatomy view.

T4a cell 13882 has a soma landmark [79438, 34824, 41092] but no assignedOIHex1/2 in this table. It remains simulated and can be inspected or directly pulsed. The default patterned encoder does not inject image pixels into it, because that encoder deliberately targets mapped L1 cells.

The anatomy view centers and scales available landmarks for display. Its canvas coordinates are not a physical ruler and are unrelated to the 32 x 24 stimulus grid.

bodyId is an identifier, not an array offset, neuron count or behavior code. “L1” is a type, while 10465 is one instance. Keep IDs as integers or strings; do not convert through a lossy floating-point representation.

A row as code, and missing values

This is a selected-field representation of a real annotation row. The full row has more fields; selecting fields for readability does not change their values.

```
{
  "bodyId": 10465,
  "type": "L1",
  "superclass": "ol_intrinsic",
  "somaSide": "L",
  "assignedOlHex1": 28.0,
  "assignedOlHex2": 22.0,
  "somaLocation": null,
  "status": "Traced"
}
```

superclass is a broad classification; `ol_intrinsic` is an optic-lobe intrinsic category. **status** records a curation state. It does not certify that our simulation parameters are correct. **somaLocation** gives a landmark when known; a missing landmark is not evidence that a cell lacks a cell body.

The importer preserves the annotation table's Arrow schema, including ordered dictionary metadata. Recreating it carelessly from plain Python rows can change types or dictionary ordering. The examples in this book are for explanation; the retained `neurons.feather` file is the structured source used by the atlas.

Small inspection command

```
from pathlib import Path
import pyarrow.feather as feather
p = Path('data/malecns-graph-v1/neurons.feather')
t = feather.read_table(p, columns=['bodyId', 'type', 'somaSide'])
print(t.slice(0, 5).to_pylist())
```

Run this inside the installed environment after the real dataset has been prepared. It does not fetch the dataset or start a simulation.

Neurotransmitters are annotations

The transmitter table uses **body** as its join key, corresponding to `bodyId` in the annotation table. Our importer uses **consensus_nt**. Other fields expose prediction counts, confidence, cell-type aggregation and a `ground_truth` annotation where available. A field named `ground_truth` is upstream metadata; our project has not independently measured that cell's physiology.

body	type	predicted_nt	consensus_nt
10001	DNp01	acetylcholine	acetylcholine
10465	L1	glutamate	glutamate
13882	T4a	acetylcholine	acetylcholine
18251	L1	glutamate	glutamate
71174	T5a	acetylcholine	acetylcholine

For L1 cell 10465, the source `predicted_nt_confidence` is about 0.849 and the consensus label is glutamate. For DNp01 cell 10001, the predicted confidence is about 0.5005 and `ground_truth` is null. These values refer to the source classification procedure; they are not the probability that a particular UI action will work.

We assign an outgoing fast sign according to a declared policy: acetylcholine positive; GABA and glutamate negative; dopamine, serotonin, octopamine, unknown and unrecognized labels zero. The same presynaptic sign is used for all of a cell's outgoing edges. This omits target receptor differences and modulatory mechanisms.

The current model makes every mapped L1 cell's outgoing glutamate influence inhibitory. That is a coarse assumption with major consequences. A biologically named input population does not guarantee biologically correct downstream responses.

Real connections become modeled weights

These are six retained incoming connections to T4a cell 13882. The contact counts come from the verified dataset; the last column is produced by our own sign and scale policy. Float32 display values below are rounded.

Sender ID	Receiver ID	Contacts	Model weight (mV)
529386	13882	32	8.800
10157	13882	22	-6.050
36766	13882	13	3.575
18045	13882	11	3.025
90615	13882	10	2.750
533488	13882	9	-2.475

```
effective_weight = contact_count * 0.275 * presynaptic_sign
32 * 0.275 * (+1) = +8.800
22 * 0.275 * (-1) = -6.050
```

A positive number changes the receiving cell's synaptic state upward when the sender's delayed spike arrives; a negative number changes it downward. It does not mean the whole receiving cell is permanently excitatory or inhibitory. The cell may receive both kinds of inputs.

The connection table's **weight** column is an anatomical contact count. Our prepared **weights.npy** contains modeled signed increments. **anatomical_contacts.npy** keeps the original counts in matching edge order. Never describe the latter as trained weights or the former as measured electrical conductances.

A directed edge can summarize several synaptic contacts. Saying "25.6 million connections" and "124.2 million contacts" describes different levels of aggregation, not a discrepancy in the same count.

From raw segments to retained neurons

Our population rule retains annotation rows whose superclass is neither empty nor null, including uncertain tbc classifications. It then retains each connection only if both endpoints are in that set. There is no pruning by a minimum contact count. Nodes with no retained edges stay in the graph.

Quantity	Observed value
Raw annotation objects	211,577
Retained nodes	166,700
Excluded annotation objects	44,877
Raw directed segment rows	151,856,684
Retained directed neuron-pair edges	25,582,938
Contacts represented by retained edges	124,177,617
Retained isolated nodes	217
Edges with zero fast modeled weight	1,113,526

“Full retained graph” therefore means all edges between nodes retained under this particular rule. It does not mean that every raw segment, every microscopic contact or every molecular process in the living animal is present. The inclusion rule belongs beside any reported result.

Initial overview pages and older preprints can show different totals. For this project, use the data version and audited import record, not a rounded headline or a count from a different female-brain reconstruction. MaleCNS IDs must not be mixed with FlyWire IDs without an explicit cross-dataset mapping.

Evidence: the supplied dataset sample record and prepared graph.json; the importer rejects duplicate retained IDs, nonpositive contact counts and duplicate neuron-pair rows rather than silently resolving them.

Arrays, sparse graphs and integrity

A dense 166,700 by 166,700 matrix would reserve entries for about 27.8 billion possible ordered pairs. Most are absent. We use compressed sparse rows (CSR): only retained edges are stored, grouped by the neuron receiving them. This is a storage choice, not a change to the anatomy.

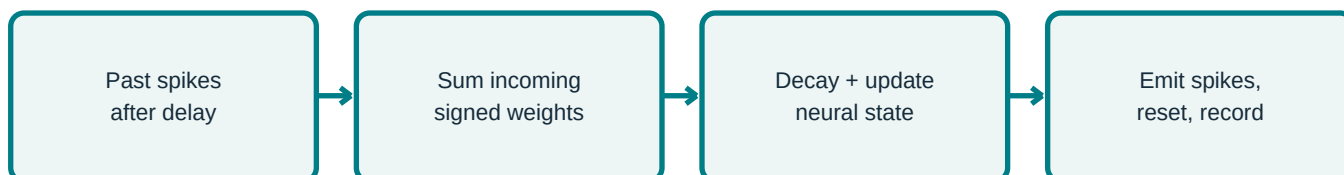
File	Meaning
ids.npy	int64 body IDs; array position is the internal index
indptr.npy	int64 row boundaries; incoming edges for cell i are $\text{indptr}[i]:\text{indptr}[i+1]$
indices.npy	int32 presynaptic internal indices for each stored edge
weights.npy	float32 effective signed increments
anatomical_contacts.npy	Original integer contact counts in the same edge order
neurons.feather	Retained annotations aligned to ids.npy
graph.json	Counts, assumptions, source metadata and file hashes

With the convention $W[\text{post}, \text{pre}]$, multiplying W by a delayed spike vector computes incoming signals. Reversing that convention changes the circuit and is a serious bug. Our reference, CPU and CUDA checks test causal transmission and timing on small networks.

A SHA-256 digest fingerprints exact bytes. It helps detect accidental corruption or a changed release. A matching digest proves identity with the locked file, not anatomical correctness or trustworthiness of every upstream annotation. The download digests were pinned from observed HTTPS downloads; they are not an independently signed biological certificate.

What changes on every tick

Every neuron has a membrane value v , a synaptic state s and a refractory counter. A delayed-spike buffer stores recent output events. Each tick gathers inputs from the graph, updates the synaptic state, moves eligible membrane values and emits threshold-crossing events. The equations are deterministic for a fixed graph, input and implementation.



One model tick. The graph stays fixed; state values change.

Our tick is 1 millisecond of **simulated** time. The processor may take more or less than 1 millisecond of real time to calculate it. The browser normally receives a summary about every 250 milliseconds of wall time. A slow browser does not change the graph or manufacture spikes.

Between runs, Pause preserves current state. Continuing then evolves from that state. Reset instead returns membrane values to rest, clears delayed spikes, counts and history, and selects a blank pattern. These operations answer different questions: continuation tests ongoing dynamics; reset enables a comparison from the same initial state.

State persistence is not training. The weights remain fixed throughout this version. A response that differs after a previous pulse can be caused by lingering state, rather than acquired skill or a new memory rule.

The equations we actually execute

```
incoming_i = sum_j W[i,j] * spike_j(t - delay)
s_i = exp(-dt / tau_s) * s_i + incoming_i
v_i = v_i + (rest - v_i) * dt / tau_m + s_i + drive_i
if eligible_i and v_i >= threshold:
    spike_i = 1
    v_i = rest
    refractory_i = refractory_ticks
```

The membrane update applies only to cells eligible to integrate. During refractoriness, the counter decreases and the cell does not emit another spike. Synaptic state continues to decay and receive incoming events even during that interval; it is not reset when the cell spikes. This is our explicit recurrence, not an assertion that every published simulator uses the same convention.

$W[i,j]$ is the modeled signed edge from cell j to cell i . **$drive_i$** is the externally injected increment for this tick. The synaptic and external terms in this implementation are added directly as per-tick increments. They are not physical currents in amperes and are not multiplied by the membrane time-step ratio again.

A simple example starts at -52 mV with no synaptic input and a constant drive of 0.6 mV/tick. After one tick, v is -51.4 mV. On the next tick, leak contributes $0.05 \times (-52 + 51.4) = -0.03$ mV, so v becomes -50.83 mV. Continued drive may eventually cross -45 mV. Network inhibition can change that outcome.

Do not present this as a direct replication of Shiu et al. Different input units, integration choices, reset behavior and graph matter. Shared parameter names or constants do not make two models equivalent. [7]

Parameters, rates and time units

Setting	Current value	Interpretation
dt	1 ms	One numerical integration tick
Rest / reset	-52 mV	Starting value and post-spike reset
Threshold	-45 mV	Eligible cells emit when reaching or exceeding it
Membrane time constant	20 ms	Sets the leak fraction dt/τ_m
Synaptic time constant	5 ms	Sets exponential decay of synaptic state
Refractory interval	2 ms	Two following ticks without membrane integration
Synaptic delay	2 ms	Spikes affect targets two ticks later
Contact scale	0.275	Chosen conversion into modeled synaptic increments
Normal chunk	20 ticks	20 ms summarized per update

Hz means events per second. One spike in a 20 ms window is $1 / 0.020 = 50$ Hz for that neuron. A population rate divides the summed count by both elapsed seconds and the number of neurons in that population. Large groups therefore do not automatically win simply because they contain more cells.

The “active this chunk” percentage counts neurons that spiked at least once in that window, divided by all retained neurons. It is not the fraction of brain capacity being used. A cell can have a changing membrane value without producing a spike, and hence count as inactive by this particular metric.

The final chunk can be shorter than 20 ms. Rates use its actual duration. Simulated time, compute time and browser refresh rate are different quantities and must be reported separately.

Why model stability is a scientific gate

A solver can implement its equations correctly while those equations produce unrealistic neural states. Our checks compare numerical implementations; they do not prove that one set of global parameters is suitable for every cell in the full CNS.

An exploratory session, 20260919T172706Z-interactive-c6d167, applied a left-eye grating followed by a 1.5 mV/tick, 20 ms pulse to T4a cell 13882. At 620 ms, the last 20 ms window had a network mean rate of approximately 46.56 Hz and a mean membrane value of approximately -419.22 mV. That voltage is not credible as a biological recording. The raw result is retained and discussed in exp_1.pdf.

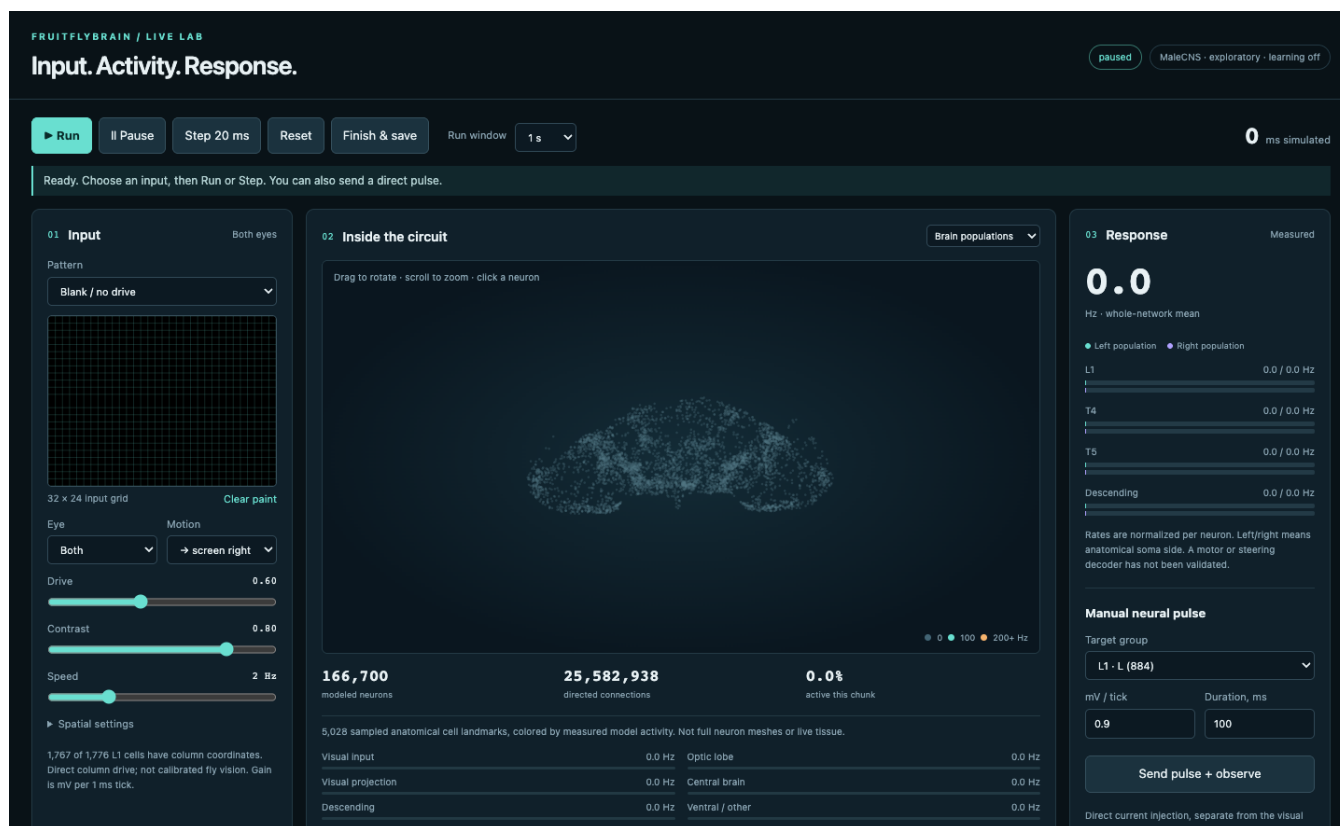
Our recurrence lacks receptor reversal potentials and conductance limits. Its synaptic state can accumulate strong inhibitory or excitatory effects. A visually impressive burst is therefore a diagnostic observation, not a successful reconstruction of natural computation. Clipping the chart or hiding unfavorable neurons would conceal the problem.

What a proper follow-up measures

Compare blank, uniform and pulsed inputs; sweep declared gains and time constants; examine voltage distributions, runaway rates, silence, recovery and sensitivity to signs. Test reduced time steps only with an explicit rescaling and equation definition. Choose development settings before freezing evaluation trials.

This handbook records the limitation rather than silently “fixing” it by changing the scientific model. Stable engineering behavior, biological plausibility and task performance require different evidence.

A screen you can read left to right



Actual MaleCNS dashboard, paused. Screenshot from the working project; colors encode model activity, not live tissue. The remaining plots and inspector appear below these panels.

The numbered panels answer three questions: what did we supply, what changed inside the model, and what response did we measure? Start with the top Run/Pause controls and simulated-time counter. Then read the Input pattern, the activity view and the Response bars in order.

The following pages explain each control individually. A blank or dim view after Reset is an expected starting point, not a missing animation. Screenshots are illustrative records of the interface; your current numbers depend on the dataset and intervention.

The three-panel workspace

The dashboard separates **Input**, **Inside the circuit** and **Response**. Read these left to right. The top transport bar controls how far the simulation advances. Lower panels show activity over time, individual spike events, a neuron inspector and saved session records.

Area	What to look for
Status badge	Paused/running/disconnected and whether the dataset is a synthetic demo
Input image	The exact discrete pattern sampled by the encoder, before multiplication by Drive
Circuit view	Locations colored by calculated spikes; a sample rather than every neuron
Response	Whole-network and selected left/right population firing rates
Plots	Rate history and a fixed sample of spike events
Session record	Dataset, backend, run ID, simulated time and whether you are viewing saved data

First check the dataset label. The portable CPU setup starts a synthetic demo. Your main GX10 dashboard uses MaleCNS. Seeing an L1 or T4 label in the demo does not make its invented graph anatomical. The experiment record also identifies the selected dataset and backend.

The display is a browser client. The server runs the equations and saves records. Closing a browser tab does not automatically reset the server. Closing the server terminal stops that process; a separate SSH tunnel only carries the connection.

All displayed activity comes from the model. The rotating anatomy view is not an animation of thoughts, conscious experience or live tissue. Nothing in the current interface reads a natural motor intention.

Run, Pause, Step, Reset and Save

Button or option	Exact behavior
Run	Advance for the selected additional window: 0.5, 1, 2 or 5 seconds of simulated time. It then pauses.
Pause	Stop further advancement at a chunk boundary. Keep state, input, queued pulse timing and existing record.
Step 20 ms	Advance 20 ticks and pause. It is one calculation chunk, not one spike.
Reset	Complete the current record, return the model to rest, clear history and choose blank input. The selected inspector cell may remain selected.
Finish & save	Complete and close the current record. A subsequent Run starts a fresh model state with the currently selected pattern.
10-second session limit	Automatically complete the record at 10,000 ms. Further Run starts a new session.

A run directory is created when the model first advances, not merely when you move a slider. The chosen initial stimulus is recorded. Later changes are stamped with the simulation tick when the server applies them. Each completed session is a separate record; Reset does not erase earlier evidence.

Pausing does not consume the remaining duration of a pulse in wall time. Pulse duration is measured in simulation ticks. If you pause during a pulse and later continue, its remaining scheduled ticks can still occur.

Controls act on the one server session. Two browser tabs attached to the same service are not two independent brains. Use separate processes, ports and run folders for genuinely independent simultaneous sessions.

Pattern choices and the input image

Pattern	What the encoder receives
Blank / no drive	All zero image values; no patterned external input
Uniform light	All one image values; each mapped selected input receives Drive each tick
Moving grating	Sinusoidal stripes whose phase changes with simulated time
Spot	A smooth Gaussian patch centered at the selected position
Paint	Your persistent 32 x 24 array of values, drawn with the pointer

The grayscale image represents values between 0 and 1. Black is 0 and white is 1. For each mapped L1 neuron, the software chooses the nearest grid address and multiplies that value by Drive. A white pixel and Drive 0.6 deliver a +0.6 mV increment per eligible model tick at that input address; they do not deliver a measured intensity of physical light.

Choosing Blank does not instantly force all neurons to rest. Previously accumulated synaptic state and delayed spikes can continue to produce activity. Reset followed by Blank is the appropriate baseline when you want to start with no history.

Pattern selection changes the input configuration; while paused it does not advance the neural state. You can inspect a preview and then press Step or Run. A grating only changes phase as simulated time advances.

The screen is an experimental stimulus generator. It is not a camera feed, a natural scene recognizer, a reconstructed view through fly optics or an image of what the fly consciously sees.

Every input slider and selector

Expand **Spatial settings** to access Cycles and Spot radius. Other controls are visible in the Input panel.

Control	Range / choices	Meaning
Eye	Both, Left, Right	Which mapped L1 cells receive the image; Both receives the same pattern twice through separate maps
Motion	Screen right / left	Sign of the grating phase motion in screen coordinates
Drive	0 to 1.5, step 0.05	Maximum patterned increment, in mV per tick
Contrast	0 to 1, step 0.05	Grating modulation around a mean of 0.5
Speed	0 to 8 Hz, step 0.25	Grating temporal cycles per simulated second
Cycles	1 to 12	Grating spatial cycles across image width
Spot radius	0.02 to 0.5	Gaussian standard deviation in normalized image coordinates

Contrast and Speed affect the grating; changing them does not change a painted pattern or the uniform image. Cycles controls how many repetitions fit horizontally. The spot uses its radius and position; its tails fade smoothly and are not cut off at that radius.

At Contrast 0, the grating is a constant 0.5 image, not a black screen. At Speed 0, its stripe pattern is stationary. At Drive 0, every image produces zero patterned current, even if the preview remains bright. These are useful controls for separating visual configuration from neural input.

For the grating, intensity is $0.5 \times [1 + \text{contrast} \times \sin(2 \pi (\text{cycles} \times x - \text{direction} \times \text{frequency} \times \text{time}))]$. Time is measured in seconds here. Increasing temporal frequency with fixed cycles changes stripe speed; neither parameter is a physical fly-eye angular velocity until calibrated.

Drawing and placing a spot

Paint a manual pattern

Choose Paint and drag across the input canvas. The pointer sets visited cells to white; dragging connects positions so a fast motion need not leave isolated gaps. Right-drag erases cells on devices that support that gesture. Clear paint sets the entire array to zero and keeps Paint selected. The software holds this image until you edit it again.

Your drawing is a spatial pattern of drive, not a semantic command. Writing “hello” does not create a language input channel; it creates bright and dark addresses. Drawing an arrow does not tell the simulator to move in that direction. You can ask whether two drawings produce different neural responses, but that is a much narrower question than recognition.

Place or move a spot

Choose Spot and click the canvas. Clicking the canvas while another non-Paint pattern is selected also switches to Spot. The x and y positions are clamped inside the input square. Outside form fields, arrow keys move the spot in steps of 0.05 normalized units. If another pattern is selected, arrows choose a grating and its left/right motion direction.

A useful comparison

From Reset, paint a vertical line, Step several times and record the resulting activity. Reset again, paint a horizontal line of comparable brightness and repeat with the same Drive and duration. Differences establish sensitivity under this encoder. They do not yet establish orientation-selective neurons or calibrated biological vision.

For fair comparisons, keep input energy, eye, duration and initial state explicit. An image with more white pixels generally stimulates more addresses; that alone can change network activity.

Manual neural pulses

A pulse bypasses the image mapping and injects a constant increment into a target set for a specified duration. It is a controlled perturbation of model state. This resembles the logic of asking “what happens if these cells are activated?”, but is not a calibrated optogenetic experiment or a physical electrical-stimulation protocol.

Control	Meaning
Target group	A listed population and soma side; the number is its membership count
mV / tick	Signed amplitude from -2 to +2; positive adds to the update, negative subtracts
Duration, ms	1-500 whole simulation ticks
Send pulse + observe	Schedule the pulse and advance for its duration plus 100 ms of observation, within the session limit
Pulse selected neuron	Apply the same amplitude/duration to the inspected body ID only

Pattern input remains active during a pulse. The two drives add where their targets overlap. Choose Blank first to isolate the direct pulse. A positive pulse does not guarantee spikes: membrane leak, refractoriness, insufficient amplitude and network inhibition still matter. A negative pulse changes the model's drive; it is not an exact biological silencing method.

The group count matters. Pulsing 884 L1-left cells is a different intervention from pulsing one T4a cell. Group selection uses available labels and soma sides; missing side annotations mean some cells cannot belong to those left/right groups. A very small R1-R6 group is not evidence that the real eye has only that many photoreceptors.

Use a new reset state for each comparison. Pulses do not reward the network and do not update weights. The amber pulse-status line reports currently scheduled direct pulses, not learning progress.

Reading the circuit view

The MaleCNS anatomy view uses 5,028 sampled soma or tosoma landmarks from available annotations. Sampling is stratified by broad population so smaller categories can remain visible. The display is not an unbiased density map. It is also not a mesh of complete axons, dendrites or all connections.

Drag to rotate the view. Scroll to zoom. Click a nearby point to inspect its body ID. The Brain populations, Whole CNS sample and Visual populations filters change which sampled points are drawn. These operations leave the simulated graph untouched. A cell can be active but absent from the display because it was not sampled or has no usable landmark.

Visual feature	Meaning
Dim point	No spike in the latest calculation window
Cyan / lighter point	A nonzero recent model firing rate
Orange point	High rate, at least 200 Hz on the displayed scale
Selection ring	The currently inspected cell if it is part of the visible sample
Population bars	Mean rate for a broad annotation-derived class

Zero spikes do not imply an unchanged membrane value. Conversely, many bright points do not imply that a task was solved. Always pair the picture with the input configuration, numerical measurements and model limitations.

In portable demo mode, all coordinates are invented and the UI says so. In real-data mode, the landmarks come from anatomy but their colors are simulated. Neither mode is a live biological recording.

Response bars, history and spike events

The large Response number is the mean firing rate across all retained nodes in the last chunk. Below it, L1, T4, T5 and Descending rows show mean rates for the left and right soma-side groups. A difference between these bars is an observable model asymmetry. It is not automatically a left/right turn command.

The Activity through time plot keeps up to 250 chunk summaries. Its Network series uses the whole-network mean. Its T4/T5 and Descending series summarize the displayed group means; groups are averaged equally in that summary. The underlying population rates and membership counts are available in the record. The vertical scale can change to fit the plotted data.

The Spike events plot shows a fixed sample of 40 cells for the current real atlas, over the last simulated second. A mark is an actual simulated spike; a row is a sampled cell index. It is not every neuron, and row order is not a sensory-to-motor transmission time. The demo has its own smaller set of sampled probes.

Why a plot can look empty

Activity can be restricted to unplotted neurons, remain below threshold, or be too small to survive rounding in a one-decimal display. One spiking input among 166,700 cells barely changes the whole-network mean. Inspect the relevant group and selected neuron before concluding that input was ignored.

Bars use visualization scales and plots summarize finite windows. Preserve the numeric records when comparing conditions; do not estimate a scientific effect from the apparent size or color of a screenshot.

Neuron inspection and saved sessions

Type a cell name such as T4a or an exact body ID into Find. Search matches IDs and type strings and returns at most 30 results. Select a result or click a sampled anatomical point. The inspector shows its type, identifier, soma side, latest membrane value, recent spike count and number of incoming edges.

“Strongest modeled inputs” lists up to six incoming edges ranked by absolute effective weight. A negative edge can therefore rank near the top. Those are stored modeled strengths, not necessarily the senders that caused the latest spike. Establishing causality requires interventions or more detailed traces.

Under Session record, View switches between Live controls and saved records. A saved view shows stored telemetry and disables simulation controls. It is a snapshot with recent history, not an interactive rewind engine. Returning to Live reconnects to the current server state; it does not restore that historical session as the running model.

Keyboard behavior

Space toggles run/pause outside form fields. Arrow keys move the spot or change grating direction as described earlier. Shortcuts are intentionally inactive while editing inputs, selects, text areas or buttons. If nothing happens, click an appropriate non-form area before using them.

The browser may display a sampled neuron's extreme voltage exactly as calculated. Such a value is a reason to examine model stability, not to infer an unusual biological mental state. Finish & save preserves the evidence.

First exercise: establish a blank baseline

Purpose

Learn the difference between a stimulus configuration and an actual advancing model, and confirm that the resting-state baseline behaves as declared.

Step	Action	What to inspect
1	Choose Live, then Reset	Time returns to 0; the image becomes blank
2	Press Step 20 ms once	Time advances to 20; inspect network rate and group bars
3	Press Step again	Time advances to 40; inspect the raster
4	Find a neuron and select it	Its value should stay at rest under this zero-input baseline
5	Finish & save	A completed record appears in View

For this deterministic model, all cells start at rest, there is no background noise and Blank supplies no drive. A genuinely fresh blank run should therefore have zero spikes. If it does not, verify that you reset, have no active pulse, are viewing Live rather than a recording, and are attached to the intended server.

This is an **implementation expectation**, not a prediction that real flies have silent brains in darkness. Living nervous systems have ongoing activity and internal dynamics that our chosen baseline does not reproduce.

The synthetic demo is appropriate for learning this workflow. Repeat on MaleCNS if you want an actual data-backed engineering record. Always record the dataset identity alongside the result.

Second exercise: a moving pattern

Purpose

Check that a controlled change in image input reaches the intended cells and produces measurable model activity.

Step	Action
1	Reset; choose Moving grating and Left eye
2	Use Drive 0.60, Contrast 0.80, Speed 2 Hz and Cycles 4
3	Select a 0.5 s run window and Run
4	Inspect L1-left and L1-right rates, the time plot and one L1 neuron
5	Finish & save, noting the complete run ID

The image varies over time and mapped left L1 cells receive drive. Right L1 cells receive no direct patterned input in this condition, but the network can still influence them indirectly. Do not treat indirect responses as a failure of eye selection. Our direct-drive unit check, rather than a population-rate guess, verifies which addresses receive external values.

Now perform a second run from Reset with Right eye and otherwise identical settings. A difference can arise from membership counts, normalization, wiring, missing data or model assumptions. The result does not isolate natural left/right preference. For a direction comparison, hold eye fixed and change only Motion.

These are exploratory comparisons. A short two-run exercise is not the frozen Experiment 1 evaluation, does not establish a p-value, and does not satisfy the retinal-map or model-stability gates.

Third exercise: paint, probe and pulse

Begin with Reset and choose Paint. Draw a small bright patch rather than filling the entire screen. Select one eye, keep Drive fixed, and Step five times. That creates 100 ms of simulation, while letting you see intermediate updates. Finish and record the session.

For a direct-neuron test, Reset again and keep Blank selected. Search T4a; choose one explicit body ID. Set pulse amplitude to 0.9 and duration to 20 ms, then press Pulse selected neuron. This advances 120 ms unless the session limit truncates it. Examine the selected neuron, downstream activity and the fixed raster. Record a zero or weak response just as carefully as a strong response.

Compare amplitudes only in separate reset sessions. A later larger pulse in the same session is confounded with prior network state. If you want to study state dependence deliberately, say so in advance and preserve the order of interventions.

A compact experiment note

Question: Does this pulse change a chosen population?

Dataset / graph hash: ...

Source / backend: ...

Initial state: Reset + Blank

Target body ID: ...

Amplitude / duration: ... mV per tick / ... ms

Response window and metric: ...

Run ID, observations and unexpected effects: ...

Avoid renaming the outcome “reward”, “pain”, “understanding” or “learning”. In this test, the directly observable facts are the injected values and the resulting modeled voltages and spikes.

What scientific models have established

Published modeling work shows why anatomy is valuable, while also showing the need for validation. Shiu and colleagues built a connectome-based leaky integrate-and-fire model and tested specific predictions about feeding and grooming pathways with biological experiments. That supports particular sensorimotor inferences; it does not establish that an arbitrary full-brain model can play a game without engineering. [7]

Lappalainen and colleagues developed a connectome-constrained visual-system model and compared predicted activity with physiological responses. Its repository, flyvis, provides a distinct modeling framework. It is not the same dataset, equations, input geometry or training setup as our current workbench. A success in that framework does not automatically validate our uniform spiking proxy. [8]

Three tests that answer different questions

Question	Needed evidence
Does the software implement its equations?	Independent numerical comparisons, causal timing tests, input and file-integrity checks
Does it resemble real neural activity?	Measured response properties, parameter sensitivity and held-out physiological comparisons
Does the system solve a task?	An explicit task, scoring rule, controlled evaluation and competing baselines

Our current tests address mainly the first row. We have not completed the second or third. “Real wiring” is an anatomical input to the project, not a substitute for those tests.

A useful outcome may be a failed prediction that identifies a missing mechanism. Reproducible negative results are part of the research record, not something to hide behind a more attractive visualization.

Games and robots: read the whole system

The Frankweb33 robot bridge repository currently describes a working small mock backend, optical-flow processing and a motor-interface scaffold. Its MaleCNS backend is an unimplemented integration target, and it does not report a validated physical full-connectome robot. The linked social post is not a substitute for inspected code and reproducible hardware evidence. [17]

The current DOOMFLY README describes a MaleCNS-based game interface and an experimental dopamine-gated rule on 4,184 existing connections. It also reports that the v6 candidate failed visual, conditioning and survival gates. Older fixed-weight review pages describe a previous configuration; they should not be cited as proof that the current project has no plasticity. Weight changes alone still do not demonstrate learned competence. [18]

DOOM-x-Fly reports 57 exits in 100 held-out runs on one Doom level, using a fixed filtered connectome model plus a 5.4-million-parameter trained decision head. The readout was trained from a privileged navigator. This is an author-reported hybrid result, not independently reproduced here, and it does not establish superiority of fly wiring over matched alternatives. [19]

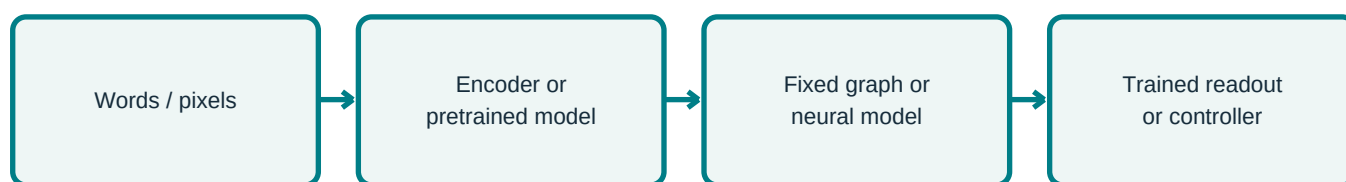
Fly64 connects a fly-derived model to a game interface with engineered control assignments. Its documentation explicitly separates visible game actions from reward-based learning. A video of movement can demonstrate an interface without demonstrating a learned objective. [20]

For any demo, ask: what actually runs; what sees the pixels; what produces actions; which parameters change; what comparisons were performed; and what evidence is independently available? Record the reviewed version and date.

Language, pretrained AI and adapters

A language-capable hybrid may combine a connectome-derived numerical network with an already trained language model. In FLM, a pretrained language backbone and the graph remain fixed while a 278,528-parameter adapter learns a correction to token scores. The repository explicitly attributes language ability to the pretrained model and reports no established advantage from fly anatomy over its matched direct-input control. [21]

That is different from a fly acquiring grammar. Token embeddings are engineered numerical inputs, the graph update is an abstract recurrence, and a conventional model already contains language competence. Other interfaces may simply map recognized words to stimulation commands and print prepared explanations.



A possible hybrid architecture. Our current project has no language encoder or trained readout.

Where learning could live

Training only an external readout leaves connectome weights unchanged. Fitting cell parameters changes the dynamical model without necessarily implementing biological memory. Updating selected synapses according to a plasticity rule changes the modeled circuit itself. Every report should say exactly which of these occurred.

The current drawing canvas accepts brightness values, not words as language. Drawing letters is an image experiment. There is no hidden language model, natural-language parser or learned semantic controller in FruitFlyBrain.

Reward, memory and learning are different

Reward is a feedback quantity defined by an experimenter or environment. **Memory** requires a specified persistent mechanism. **Learning** requires a demonstrated change attributable to experience under appropriate comparisons. Calling an input pulse “reward” supplies neither the mechanism nor the evidence.

A useful future reinforcement-learning experiment would define observations, actions, an environment, a reward function, trainable parameters and an update rule. It would also define episode resets, evaluation without ongoing reward, and what counts as improved performance. A rising training score alone can reflect overfitting, changed initial conditions or a decoder adjustment.

Possible approach	What changes	Appropriate claim
Readout learning	External mapping from fixed graph state to action	A trained hybrid controller
Parameter fitting	Declared physiological or dynamical parameters	A calibrated model, if validated independently
Synaptic plasticity	A specified subset of modeled synapses	A learning mechanism in the model, if causal evidence supports it
Reward pulse alone	Temporary neural state	A perturbation, not demonstrated learning

FruitFlyBrain currently implements none of these training mechanisms. All learning indicators should remain off. Experiment 2 will be documented before enabling a concrete reward-learning rule.

The broad idea of using a fixed recurrent system and learning a readout is often called reservoir computing. A biological topology can be an interesting prior, but comparison with suitable nonbiological graphs is needed before claiming an advantage.

Mushroom bodies and a future experiment

The mushroom body is a fly brain system studied in associative learning. Kenyon cells, mushroom-body output neurons (MBONs) and dopamine-related circuits provide a useful starting vocabulary. The connectome describes many of their anatomical relationships; it does not by itself supply a complete learned association or the appropriate dynamics for every compartment. [11]

A future experiment could use two controlled sensory patterns, pair one with a defined reinforcement event, and then test both with reinforcement absent. Its model must specify which synapses can change, how timing and activity affect that change, how changes are bounded and which states persist across trials. The chosen rule must be justified and compared with appropriate biology or declared as an engineering abstraction.

Comparison	Question it addresses
Plasticity disabled	Does the claimed improvement require weight updates?
Shuffled or yoked reward	Does the feedback timing or contingency matter?
Unpaired pattern	Is the effect specific rather than global suppression?
Reset fast state but keep weights	Is there retention beyond transient activity?
Reset learned weights	Does removing the learned change remove the effect?
Matched readout / rewired graph	Is anatomy specifically useful, or is a simpler controller sufficient?

These are proposed designs, not implemented dashboard options. Negative pulse amplitude is not punishment, positive amplitude is not sugar, and a dopamine label assigned zero fast weight is not a dopamine-learning model.

Experiment 1 and the staged roadmap

Experiment 1 asks whether a fixed, explicitly defined MaleCNS-constrained model produces reproducible direction-dependent responses to controlled motion. The primary scientific output is neural response. A virtual steering display would be a secondary engineered decoder and must not be added as if it were already validated.

Stage	Current status / requirement
Engineering infrastructure	Implemented importer, solvers, UI, controlled inputs and run records
Model and mapping audit	Pending optical conventions, input polarity, dynamics and stability validation
Connectivity control	Pending tested null preserving relevant graph properties
Model freeze	Pending dated specification of graph, mapping, parameters, readout and analysis
Evaluation	Reserved trials only after the freeze; development and evaluation kept distinct
Experiment 2	Write the reward/plasticity protocol before implementation

The existing exp_1.pdf defines the detailed trial plan, response windows, controls and analysis. Its original prospective content remains distinguished from the later engineering observations. This handbook teaches the concepts and controls; it does not silently replace the frozen-protocol requirements with a set of favorable demo runs.

The two GX10s can run independent matched conditions or replicas after the scientific choices are settled. Their interconnection does not currently make one distributed simulation. The present live server runs on gx10-a; gx10-b has synchronized source and a checked runtime for later work.

The next scientific priority is stability and input/readout validation. Adding rewards to unstable dynamics would make it harder to identify the source of an apparent improvement.

Choose a device and an execution mode

Device / environment	Supported route	Limit
Mac, Apple Silicon or Intel	CPU mode with Python 3.11-3.13 and matching dependency wheels	Apple GPUs are not used by this implementation
Linux x86-64 / ARM64	CPU mode; CUDA mode on supported NVIDIA systems	CUDA also needs a compatible toolkit/compiler and driver
Windows	Use a suitable WSL2 Linux environment for setup.sh	Native Windows installation is not validated
Phone / tablet	Browser client to a separately running service through a trusted private tunnel	No supported on-device simulator installation
Other small / older systems	CPU only if the OS and pinned wheels are supported	Not every device has adequate RAM, disk or Python packages

“Portable” means the code has a CPU path and avoids personal SSH aliases for ordinary setup. It does not mean unlimited performance on arbitrary hardware. Start with the synthetic demo to verify the interface; opt into the real dataset separately.

For a full-data workstation, plan for at least 16 GB RAM, preferably 32 GB, and at least 6 GiB free disk for initial preparation plus additional space for records. These are planning recommendations, not a guarantee for every OS. Raw import has a higher peak-memory footprint than the final graph files.

Mac CPU and Linux ARM64 GX10 paths are tested in this project. Intel Macs, Linux x86-64 and WSL are intended compatible targets subject to dependency availability; they are not all independently hardware-tested.

A first installation that starts the UI

Install Python 3.11, 3.12 or 3.13 with venv and pip. Download or clone this project, open a terminal in its directory, then run:

```
sh setup.sh
```

The default creates a project-local .venv, installs pinned packages, verifies the CPU solver, prepares the synthetic demo and starts a loopback server. Open <http://127.0.0.1:8765/> in a browser on that computer. Keep the terminal open. Ctrl+C stops the server. Your records remain on disk.

If port 8765 is already used, choose another port. The installer will not kill an existing process:

```
sh setup.sh --port 8876
```

If a compatible Python is installed outside PATH, provide the interpreter explicitly:

```
FFB_PYTHON=/path/to/python3.12 sh setup.sh
```

On a supported Linux distribution, missing venv or pip support may require your OS's Python-venv package. On macOS, a normal Python.org or package-manager installation can supply the interpreter. This script does not use sudo, install an OS package manager or alter system Python.

At the time of this handbook, the project has not been published to a Git repository. There is intentionally no invented clone URL. Once a repository is published, the same setup.sh works from that checkout; the runtime does not require Git metadata.

Run the real data or select CUDA

Real MaleCNS on CPU

```
sh setup.sh --dataset male-cns --backend cpu
```

This explicitly opts into the download and import. Existing verified files are reused. The importer does not overwrite an existing prepared graph; a conflicting or incomplete graph should be diagnosed rather than silently replaced. Full-graph CPU runs can be much slower than simulated time.

Real MaleCNS on an NVIDIA Linux machine

```
sh setup.sh --dataset male-cns --backend cuda
```

Install the NVIDIA driver and an appropriate CUDA toolkit first. The build detects the first visible GPU's compute capability and passes the matching architecture to nvcc. FFB_CUDA_ARCH can override that choice, and FFB_NVCC can select a compiler. The toolkit must actually support that architecture. CUDA is not available on a Mac through this backend.

Other useful setup options

```
sh setup.sh --backend auto --no-start
sh setup.sh --venv /path/to/environment --no-start
sh setup.sh --data-dir /path/to/data --runs-dir /path/to/runs
```

Auto chooses CUDA only when the required Linux tools are discoverable; otherwise it chooses CPU. An explicit CUDA request that cannot work fails with an actionable message. No-start prepares and checks the installation without starting a persistent service, then prints a start command.

Do not assume all NVIDIA devices behave like GB10. Compiler support, memory, driver compatibility and performance must be checked on the target machine. The numerical doctor check runs before the server starts.

What setup.sh changes and preserves

Location	Purpose
.venv/	Isolated interpreter environment and installed dependencies
data/demo-v1/	Deterministic invented teaching graph
data/malecns-v1/	Downloaded raw files and checksums, only when requested
data/malecns-graph-v1/	Prepared real graph and annotations
build/	Optional native CUDA library and build record
runs/local/	Default local session records
runs/local/_sources/	One exact source archive per local run source identity

The environment pins NumPy 2.2.6, PyArrow 19.0.1 and SciPy 1.15.3. Setup uses binary dependency wheels to avoid silently launching long source compilations. A supported Python version and compatible wheels are prerequisites. The package is installed from the checkout in editable mode, so keep that checkout in place.

A dependency-install stamp lets repeated setup reuse an unchanged environment. Dataset digests are checked before reuse. A rerun does not delete data, completed runs or another server. If a path already contains the wrong kind of dataset or an invalid environment, setup reports the conflict.

After editing code, restart the service before recording new sessions. A running Python process has already imported code. Source preservation rejects inconsistent new archives rather than pretending newly edited files were the code that produced old state.

Uninstalling is a file-management decision: stop the service first, then remove the chosen environment or data directories only if you no longer need them. Keep run records and source archives if you want to reproduce earlier observations.

Mac display, remote computation

On this project's two GX10s, the Mac keeps authoritative source, each machine has a verified versioned release and the real live graph runs on gx10-a. Remote data and runs live outside release folders, so deploying new code does not delete experimental evidence.

```
python3 scripts/remote.py deploy gx10-a
python3 scripts/remote.py deploy gx10-b
python3 scripts/remote.py bootstrap gx10-a
python3 scripts/remote.py collect gx10-a
```

Those commands use this owner's configured SSH aliases. Other users do not need them for local setup. For their own remote machine, install the project there using setup.sh, then forward the loopback service from a client computer with an appropriate SSH host and local port:

```
ssh -N -L 8876:127.0.0.1:8765 your-server
```

Open <http://127.0.0.1:8876/> on the client. The server still computes on the remote machine. The tunnel forwards traffic; it does not transfer the graph into the browser. Keep both server and tunnel alive. A phone or tablet needs an appropriate private access/tunnel arrangement; merely entering the Mac's localhost URL on a phone cannot reach the Mac.

The server binds to loopback and uses a per-service control token plus origin checks. It is a local lab service, not a hardened public multiuser application. Do not change it to a public listening address merely to make a URL convenient.

Code map and reproducible records

Component	Responsibility
datasets.py / importer.py	Pinned downloads, data joins, inclusion policy and graph construction
graph.py / atlas.py	Exact graph arrays, annotations, groups, column map and display sample
model.py / cpu.py / cuda.py	Small reference, sparse CPU solver and native CUDA wrapper
native/lif.cu	CUDA recurrence implementation
stimuli.py / live.py	Patterns, pulses, queued control and bounded simulation sessions
server.py / ui/	Loopback API and browser interface
setup.sh / scripts/setup.py	Portable dependency setup and startup
PLAN/ / docs/research/	Current PDFs / editable sources, evidence and builders

Each interactive session records `manifest.json`, `source_manifest.json`, `commands.jsonl`, `frames.jsonl`, `telemetry.json`, `metrics.json` and `spike_counts.npy`. The manifest identifies parameters, mapping, groups, sampled cells, environment and graph. Commands include their application tick; initial stimulus is stored explicitly. Metrics include an input-byte digest.

`frames.jsonl` stores each chunk's summaries and sampled raster events. `telemetry.json` is the latest display snapshot; it is not the entire full-neuron trajectory. `spike_counts.npy` stores total counts for every retained node. Full continuous voltage and spike histories are not retained for every neuron, so some retrospective analyses require a new instrumented run.

Source archives and fixed inputs make reconstruction of a run possible, but the UI has no one-click historical-state restore. Reproduction must honor exact command timing and reset semantics. Numerical differences between backends or hardware must be checked rather than assumed absent.

What has actually been tested

Check	Observed outcome
Fresh Mac checkout	CPU setup, numeric doctor check and repeated setup passed, including a path with spaces and invocation from another directory
Browser controls	13 end-to-end controls and layout checks passed on the CPU demo and real-data CUDA dashboard, including Reset acknowledgement under delayed responses
Mac full retained graph	100 ms of real-graph engineering simulation completed on an Apple M5 Pro with 24 GiB RAM
Numerical and API suites	19 tests passed on Mac (CUDA test skipped); all 20 passed on each GX10, including native CUDA trajectory comparisons
Fresh Linux checkout	Independent setup in a temporary path with spaces passed for CUDA and CPU on GX10-b; no personal checkout or SSH aliases needed by setup
Unverified hardware	Other CPU/OS and NVIDIA combinations still need checks on those devices

The Mac full-graph run 20260919T175111Z-be16621d simulated 100 ms in approximately 1.640 seconds of measured execution, plus setup and telemetry outside that timing. It recorded 263,861 spikes under its declared direct-stimulation protocol. This was a short engineering benchmark, not a vision or learning result.

The rate was about 0.061 simulated seconds per execution second for that run. Do not extrapolate it into a guarantee for longer trials, different inputs or another machine. The earlier GX10 benchmark used a different duration and is not a controlled CPU/GPU speed comparison.

Installation testing demonstrates that another checkout can run without the original project path or personal SSH aliases. It does not certify every possible device, OS version, memory size or graphics driver.

Installation and connection troubleshooting

Symptom	Likely cause / next step
No compatible Python found	Install Python 3.11-3.13 with venv. Set FFB_PYTHON if the executable is outside PATH. Python 3.14 is not within this pinned environment's supported range.
No matching binary wheel	Your Python/OS/architecture combination is unsupported by a pinned dependency. Use a compatible interpreter/OS rather than silently changing scientific dependencies.
Port is in use	Another server or SSH tunnel owns it. Choose --port 8876 or another unused port; do not kill unrelated processes.
CUDA compiler or architecture error	Check nvcc, driver and device support. Use --backend cpu for the portable route.
Browser says disconnected	Check the server terminal, correct port and any SSH tunnel. A webpage cannot compute while its backend is unavailable.
Loading takes time	The real graph must be verified and loaded. Inspect terminal output before repeatedly restarting it.
Out of memory during import	Close unnecessary work, use a larger machine or begin with --dataset demo. The prepared graph's disk size understates import peak RAM.

Run `sh setup.sh --no-start` to separate installation/data preparation from server startup. The doctor command checks a tiny graph with the chosen solver. Passing that check is useful for diagnosis but does not measure full-graph speed or validate the neuroscience.

```
.venv/bin/python -m fruitflybrain.cli doctor --backend cpu
```

On a CUDA Linux host, replace `cpu` with `cuda` after building the library. The command exits with failure if its numerical check fails.

Input and result troubleshooting

Symptom	How to reason about it
Bright image, no response	Check Drive, Eye, paused time, selected dataset and active population. Brightness is a preview, not a guarantee of suprathreshold drive.
Network reads 0.0 Hz	Rounding or dilution over many neurons can hide a small response. Inspect the relevant group and a neuron.
T4/T5 stay quiet	The input mapping and dynamics are exploratory; a named type does not guarantee its known biological response.
Activity continues after Blank	Prior synaptic and delayed state can remain. Reset for a history-free baseline.
Two runs disagree	Compare initial state, pulses, command ticks, data/source hashes, mapping and backend before interpreting biology.
Extreme negative voltages	A known model-stability limitation; preserve evidence and analyze assumptions.
Buttons are disabled	A saved record is selected, the server is loading, or the connection failed. Return to Live and inspect status.
A searched neuron is not visible	Search covers retained cells, while the point cloud is only a sample with available landmarks.

For a bug report, include the run ID, source identity, graph identity, backend, command sequence, error text and expected versus observed behavior. Avoid reporting only a screenshot of a colorful response; the numeric records are what make diagnosis possible.

If code changed while the service was running, restart it before a new session. A mixed code/state record is not reproducible. Existing completed records should be retained with their original source archive.

Frequently asked questions: biology

Is there a real fly in my computer?

No. There is a numerical simulation informed by anatomical data. The original tissue reconstruction and our executable model are different objects. No claim about consciousness or subjective experience follows from the UI.

Is the connectome a ready-trained neural network?

It is a reconstructed wiring map. Its structure was produced by biological development and the animal's history, but the file is not a generic pretrained task policy. Our software adds equations and interfaces before it can run.

Does it need training to respond?

No. Fixed equations can propagate signals through fixed wiring immediately. A response is different from learning a new task. Our current experiment studies those fixed responses.

Does more activity mean better performance?

No. It can mean stronger stimulation, more driven cells, feedback amplification or instability. A task needs a defined score, and biological fidelity needs independent measurements.

Can I infer natural behavior from a cell label?

Only with appropriate evidence. A type name is a starting point for a hypothesis, not a guaranteed natural function for the screen direction, model state or game action in your experiment.

Can these results be generalized to all flies?

Not automatically. The graph is tied to a particular reconstruction and release, and the simulation adds assumptions. Variation between animals and datasets is separate from repeating code on the same graph.

Frequently asked questions: the software

Can I type a command to make it turn?

There is no semantic command interface or motor decoder. You can draw an image, select a stimulus or inject a pulse. The current outputs are neural measurements.

Why not render every synapse in the head view?

Tens of millions of edges would be slow and visually crowded. The view displays a labeled sample of positions, while all retained edges remain in the solver. The inspector exposes a few strong incoming edges numerically.

Are the two GX10 GPUs jointly simulating one brain?

No. The current live process runs on gx10-a. Both hosts have synchronized code, but distributed multi-GPU simulation is not implemented. A second machine is useful for independent controlled runs.

Does it use my Mac's GPU?

The portable Mac route uses SciPy on the CPU. The remote route uses the GX10 CUDA backend. There is no Apple Metal/MPS neural solver in this project.

Does Save store a complete brain checkpoint?

No. It stores provenance, commands, input identity, summaries, sampled spikes and total counts. It is not a full in-memory checkpoint with every membrane and delay-buffer value. Saved View does not restore a running historical state.

Can I start several independent experiments?

Use separate server processes, ports and run directories, sized for available memory. Multiple tabs on one server all control the same session; they are not independent replicas.

Frequently asked questions: claims and plans

Why include a synthetic demo at all?

It makes first installation small and hardware-independent enough for many laptops. It is useful for learning controls and finding software problems. The prominently labeled real-data option is required for MaleCNS-derived observations.

Why retain an unrealistic result?

Because it is evidence about the model. Deleting it or displaying only a successful-looking interval would obscure the stability problem and invite false conclusions.

Can we teach it a game or language later?

We can design a hybrid learning system, but must state which parts learn and what supplies competence. Existing conventional AI components may account for much of the result. We should first define a small controlled task and necessary comparisons.

What would establish that wiring matters?

Compare the intact graph with appropriate rewired, disconnected or alternative-architecture controls, while matching interfaces and trainable capacity. A topology-specific effect is a stronger claim than merely showing that a graph carries input information.

What is the next decision before learning?

Choose and validate stable dynamics and a defensible input/readout mapping. Then write a dated learning protocol with a specific plasticity or readout update, controls, evaluation and failure criteria.

This book explains the current implementation and the reasoning behind its limits. It does not fill scientific unknowns with confident guesses. Questions without supporting evidence remain explicitly open.

Glossary: cells, circuits and data

Term	Meaning
Anatomy	Physical structure; here, reconstructed cells and contacts
Annotation	A label or metadata field attached to an object
Axon / dendrite	Neuronal branches associated with output/input organization; real fly geometry is more complex than a simple cartoon
Body ID	Identifier for a reconstructed object within one dataset version
Cell type / instance	A named category / a particular cell in that category
CNS	Central nervous system: brain plus nerve cord
Connectome	A map of connections at a specified scale and completeness
Contact / edge	An individual reconstructed connection site / an aggregate ordered pair in our graph
Descending neuron	Cell class sending pathways toward the nerve cord; not automatically a game-action command
EM	Electron microscopy, used to image fine structure
Lamina / L1	An early visual region / a named neuron type associated with it
Neuropil	A region rich in neuronal processes and synaptic connections
Optic lobe	Major visual-processing part of the fly brain
Soma / to soma landmark	Cell-body or associated source landmark used for display when available
T4 / T5	Visual neuron families with biological motion-related roles
VNC	Ventral nerve cord; body-associated CNS circuitry

Definitions here prioritize reading this project's interface. They are not a complete taxonomy of fly neuroanatomy. Biological context: [1,9,13,14].

Glossary: dynamics and experiments

Term	Meaning
Backend	The code that executes dynamics: CPU or CUDA
Chunk / tick	A summary/calculation block / one numerical time step
Drive	An externally applied per-tick increment
Encoder / decoder	Mapping input into neural values / mapping neural output into another representation or action
Excitation / inhibition	Effects tending to increase / decrease spiking; modeled here with signed increments
Hz	Events per second; rate estimates depend on the observation window
LIF	Leaky integrate-and-fire, a simplified neuron model
Membrane potential	Electrical difference across a membrane; a model variable here
mV	Millivolt, one thousandth of a volt
Plasticity	A rule that changes a modeled connection or other persistent parameter
Raster	A plot of event times across selected neurons
Refractory interval	Temporary interval after a spike when this model cannot integrate and emit another
Reset	Return model state to its specified starting condition
Reward	Experiment-defined feedback; not itself a learning rule
Spike	A discrete output event; generated by threshold crossing in this model
Time constant	Parameter controlling a rate of decay or relaxation
Trial / replica	A specified experimental run / an independently repeated unit of evaluation

Glossary: reproducibility and statistics

Term	Meaning
Baseline	A comparison condition such as a reset blank-input run
Checkpoint	Stored state sufficient to resume a computation; our saved telemetry is not a full neural checkpoint
Confound	Another changing factor that could explain a measured difference
Control	An intervention or comparison designed to test an alternative explanation
CSR	Compressed sparse rows; a compact representation of incoming graph edges
Deterministic	Same defined inputs and implementation produce the same sequence; cross-backend agreement still needs checking
Held-out evaluation	Trials not used to tune the model or choose a checkpoint
Manifest	A structured record of what code, data and settings produced a run
Null model	A defined alternative, such as a rewired graph, used for comparison
Preregistration / freeze	An advance record of the test and choices; our local prospective document is not an external registration
Seed	A value defining random choices; engineering seed 9000 is distinct from planned evaluation seeds
SHA-256	A file-identity digest; useful for integrity, not proof of scientific truth
Source archive	Exact code files preserved for a run's source identity
Validation	A check against a specified criterion, which must be stated
Wall time	Elapsed real-world time, distinct from simulated time

Additional annotation fields

The real annotation table contains more metadata than our solver consumes. This inventory groups the remaining names so you can recognize them when inspecting Feather files. Their full upstream meanings belong to the release's annotation conventions; our model does not infer new physiology from them.

Field family	Examples in the source	Use in this project
Core selection / labels	bodyId, superclass, type, somaSide	Node identity, inclusion and population grouping
Display / column mapping	somaLocation, toSomaLocation, assignedOIHex1, assignedOIHex2	Sampled display landmarks and exploratory L1 addressing
Classification	class, subclass, supertype, group, instance, synonyms	Retained metadata; not additional dynamics parameters
Cross-dataset names	flywireType, hemibrainType, mancBodyid, mancGroup, mancType, vfbId	Do not treat these as interchangeable body IDs
Curation	status, statusLabel, matchingNotes	Retained for audit; not solver-confidence scores
Development / organization	birthtime, itoleeHI, trumanHI, somaNeuromere, serialMotif, mancSerial, mcnsSerial	Retained labels; this simulator does not model development
Other biological labels	dimorphism, fruDsx, rootSide, entryNerve, exitNerve, receptorType	Available metadata; receptor-resolved transmission is not implemented

The mere presence of a field such as receptorType or fruDsx does not mean the simulator implements that biological mechanism. Inspect code and parameter use before making such a claim.

References 1

[1] Berg et al. Sexual dimorphism in the complete *Drosophila* male central nervous system connectome. *Cell* (2026).

<https://doi.org/10.1016/j.cell.2026.08.015>

Primary reconstruction study. Use the final release/version; older preprint and landing-page counts differ.

[2] Google Research. A connectomics milestone: mapping the complete male fruit fly brain (3 September 2026).

<https://research.google/blog/a-connectomics-milestone-mapping-the-complete-male-fruit-fly-brain/>

Contributor overview of the reconstruction and associated resources.

[3] MaleCNS official download inventory.

<https://male-cns.janelia.org/download/>

Dataset formats, access routes and CC-BY terms. Exact local counts are independently extracted from the locked files.

[4] Google Neuroglancer.

<https://github.com/google/neuroglancer>

Open-source viewer for large volumetric datasets.

[5] Google flood-filling networks.

<https://github.com/google/ffn>

Segmentation research code, distinct from a neural simulator.

References 2

[6] Flyconnectome: MaleCNS supplemental data.

<https://github.com/flyconnectome/2025malecns>

Derived tables including visual-column assignments and cross-connectome mappings.

[7] Shiu et al. A *Drosophila* computational brain model reveals sensorimotor processing. *Nature* (2024).

<https://www.repository.cam.ac.uk/items/f6d6fd8e-ffa0-4e9f-aaca-71732c523253>

Primary paper repository; DOI 10.1038/s41586-024-07763-9. Associated code: github.com/philshiu/Drosophila_brain_model.

[8] Lappalainen et al. Connectome-constrained networks predict neural activity across the fly visual system. *Nature* (2024).

<https://github.com/TuragaLab/flyvis>

Authors' model repository; paper DOI 10.1038/s41586-024-07939-3. A different visual model from this project.

[9] Maisak et al. A directional tuning map of *Drosophila* elementary motion detectors. *Nature* (2013).

<https://www.nature.com/articles/nature12320>

Primary T4/T5 direction and ON/OFF response study.

[10] Bahl et al. Object tracking in motion-blind flies. *Nature Neuroscience* (2013).

<https://www.nature.com/articles/nn.3386>

Primary study motivating a distinction between motion and position-related behavior.

References 3

[11] Li et al. The connectome of the adult *Drosophila* mushroom body provides insights into function. eLife (2020).

<https://elifesciences.org/articles/62576>

Primary mushroom-body connectome study.

[13] Purves et al. Neuroscience: Neural Signaling. NCBI Bookshelf.

<https://www.ncbi.nlm.nih.gov/books/NBK10882/>

Background definitions of electrical and synaptic signaling.

[14] Purves et al. Neuroscience: Chemical Synapses. NCBI Bookshelf.

<https://www.ncbi.nlm.nih.gov/books/NBK11009/>

Background on pre/post direction, transmitter signaling and synaptic effects.

[17] Frankweb33/flybrain-robot-bridge.

<https://github.com/Frankweb33/flybrain-robot-bridge>

Community prototype; inspected working mock and planned MaleCNS integration, not independently validated hardware.

[18] nftechie/doomfly: current README and v6 status.

<https://github.com/nftechie/doomfly>

Community experimental plasticity and game interface; current negative gates must be distinguished from older fixed-baseline reviews.

References 4

[19] Aur1ety/DOOM-x-Fly: results and limitations.

<https://github.com/Aur1ety/DOOM-x-Fly/blob/main/docs/RESULTS.md>

Author-reported hybrid game results, trained decision head and controls. No independent reproduction is claimed here.

[20] ornata/fly: Fly64.

<https://github.com/ornata/fly>

Community game interface; distinguish movement from training and task learning.

[21] nftechie/flm.

<https://github.com/nftechie/flm>

Language-model hybrid with a trained adapter; language competence comes from a pretrained backbone.

Evidence, revisions and open questions

Edition 1.0 was prepared on 19 September 2026 after initial CUDA runs and interactive development observations. It replaces the short background overview with a beginner handbook. It records the portable CPU implementation and setup procedure, rather than implying that the initial project already supported every device.

The source evidence includes the project's code, tests, actual run manifests, the locked source files and the exact extracted examples in docs/research/evidence/dataset_samples.json. Public community-project descriptions were checked on this date and may change. Their self-reported scores are labeled as such; this project has not reproduced those external experiments.

The final Cell HTML was reviewed in the earlier research pass; the publisher PDF endpoint was not accessible through the text-fetch route. Some search-accessible landing pages contain older counts, so the handbook's numerical graph examples use our verified v1.0 extraction. The linked X video was not independently validated as a physical hardware demonstration.

Questions this implementation does not yet answer

Which visual dynamics reproduce measured L1/T4/T5 responses? How should the two eyes map onto a common physical scene? Which signs and physiological parameters are supported for each pathway? Is the intact wiring better than a matched control for the defined task? Can a specific learning rule improve held-out performance while retaining plausible dynamics? Those are future research questions, not missing buttons.

Keep exactly one current plan.pdf and one current exp_*.pdf per experiment in PLAN. Editable sources, evidence and builders live in docs/research. Source releases and run archives are separate records; no duplicate PDF export folder is needed.